

Minimum Spanning Trees & Kruskal's Algorithm

Lecture Notes

August 11, 2025

Outline

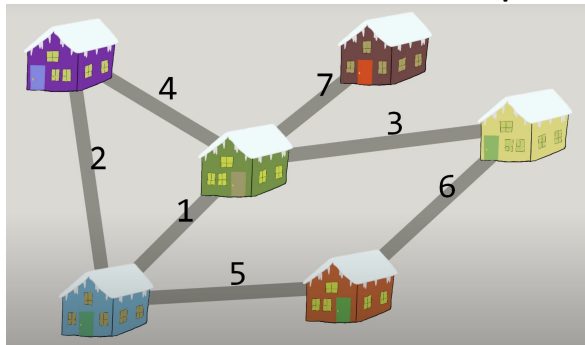
- 1 Youtube Video
- 2 Spanning Trees
- 3 Minimum Cost Spanning Tree
- 4 Kruskal's Algorithm

Video

Start by watching this video:

https://www.youtube.com/watch?v=Yldkh0a0Ecg&ab_channel=SpanningTree

How Do You Calculate a Minimum Spanning Tree? by Spanning Tree

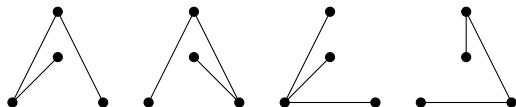


Spanning Trees

Definition: Spanning Tree

A **spanning tree** is a connected graph using all vertices in which there are no circuits.

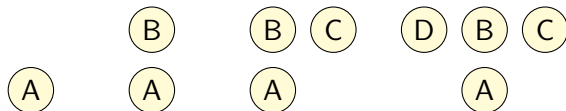
- There is a path from any vertex to any other vertex
- No circuits (cycles)
- When we have a weighted graph, we often want the **minimum cost spanning tree**



Key Property of Spanning Trees

Lemma: Number of Edges in a Spanning Tree

Let $G = (V, E)$ be a connected undirected graph with $|V| = n$ vertices. Then every spanning tree of G contains exactly ??? edges.



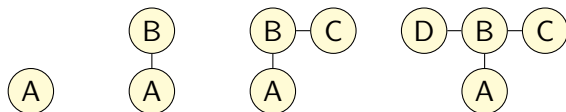
For n vertices, we need exactly ??? edges to:

- Ensure connectivity (must have at least ??? edges)
- Avoid cycles (must have at most ??? edges)

Key Property of Spanning Trees

Lemma: Number of Edges in a Spanning Tree

Let $G = (V, E)$ be a connected undirected graph with $|V| = n$ vertices. Then every spanning tree of G contains exactly $n - 1$ edges.



For n vertices, we need exactly $n - 1$ edges to:

- Ensure connectivity (must have at least $n - 1$ edges)
- Avoid cycles (must have at most $n - 1$ edges)

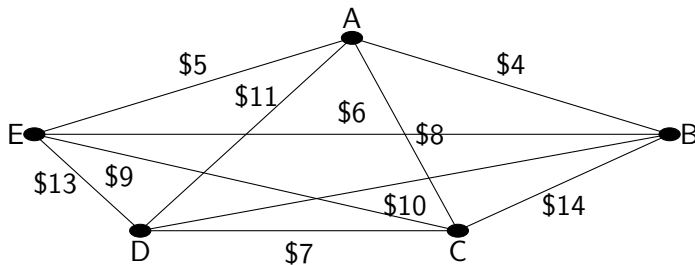
Cycles with more than $n - 1$ edges

Minimum Cost Spanning Tree (MCST)

Definition: Minimum Cost Spanning Tree

The minimum cost spanning tree is the spanning tree with the smallest total edge weight.

Practical Problem: A company needs to lease dedicated lines between their five offices (A, B, C, D, and E). The costs (in thousands of dollars per year) are shown in the graph.



Goal: Find the smallest set of edges that connects all offices while minimizing total cost.

Applications of MCST

- 1 **Network Design:** Connecting cities with fiber optic cables, water pipes, or electrical wires at minimum cost.
- 2 **Transportation Infrastructure:** Connecting towns with roads while minimizing construction costs.
- 3 **Clustering in Machine Learning:** Capturing data structure through pairwise distances.
- 4 **Communication Networks:** Ensuring connectivity between devices with minimal wiring cost.
- 5 **Approximation Algorithms:** Used as a subroutine in algorithms for NP-hard problems (e.g., Traveling Salesperson Problem).

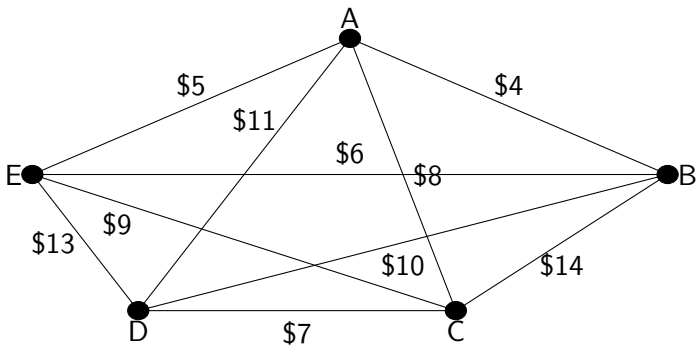
Kruskal's Algorithm

Key Idea

Build a spanning tree by **greedily** adding edges one at a time in order of increasing weight, skipping any edge that would create a cycle.

Algorithm 1 Kruskal's Algorithm

- 1: **Input:** A connected, undirected graph $G = (V, E)$ with edge weights $w(e)$.
- 2: **Initialization:** Mark all edges as unused. Set $T \leftarrow \emptyset$.
- 3: **repeat**
- 4: Select the cheapest unused edge in the graph.
- 5: **if** adding the edge would create a cycle **then**
- 6: Skip this edge.
- 7: **else**
- 8: Add the edge to T .
- 9: **end if**
- 10: **until** T contains $|V| - 1$ edges (i.e., a spanning tree has been formed).
- 11: **Output:** The set T , forming a minimum spanning tree of G .



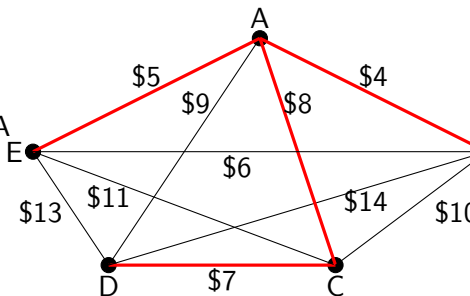
Step	Edge Considered	Forms Cycle?	Add to T?	Cost
1				
2				
3				
4				
5				
6				
7				
8				
9				

Example: Company Phone Lines

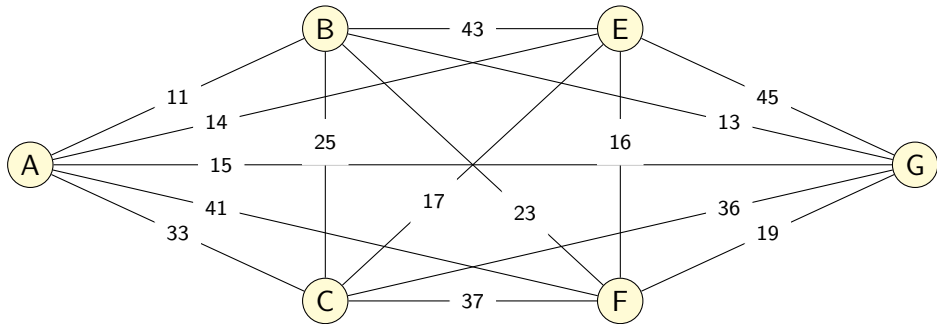
Using Kruskal's algorithm for the phone line example:

AB	\$4	OK
AE	\$5	OK
BE	\$6	reject – closes circuit ABEA
DC	\$7	OK
AC	\$8	OK

Total cost: \$24,000 per year



- Selected edges form a spanning tree (shown in red)
- All vertices are connected with no cycles
- We have $n - 1 = 4$ edges for 5 vertices
- Total cost is minimized

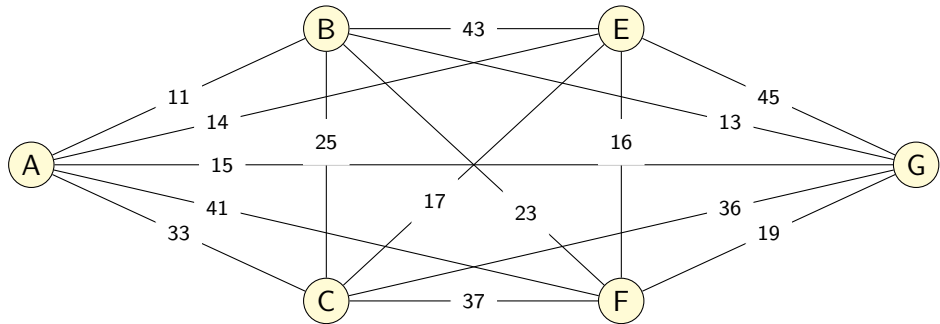


Algorithm 2 Kruskal's Algorithm

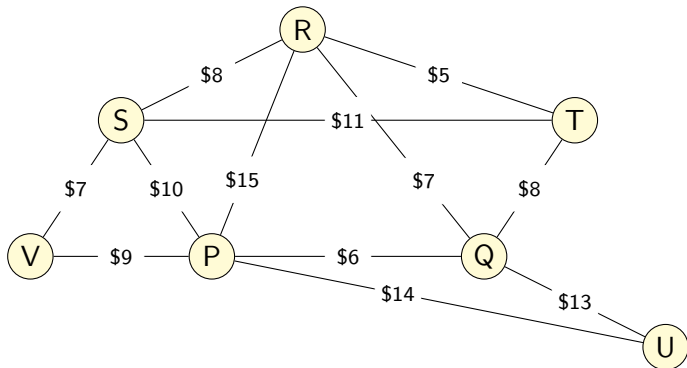
```

1: Input: A connected, undirected graph  $G = (V, E)$  with edge weights  $w(e)$ .
2: Initialization: Mark all edges as unused. Set  $T \leftarrow \emptyset$ .
3: repeat
4:   Select the cheapest unused edge in the graph.
5:   if adding the edge would create a cycle then
6:     Skip this edge.
7:   else
8:     Add the edge to  $T$ .
9:   end if
10: until  $T$  contains  $|V| - 1$  edges (i.e., a spanning tree has been formed).
11: Output: The set  $T$ , forming a minimum spanning tree of  $G$ .

```



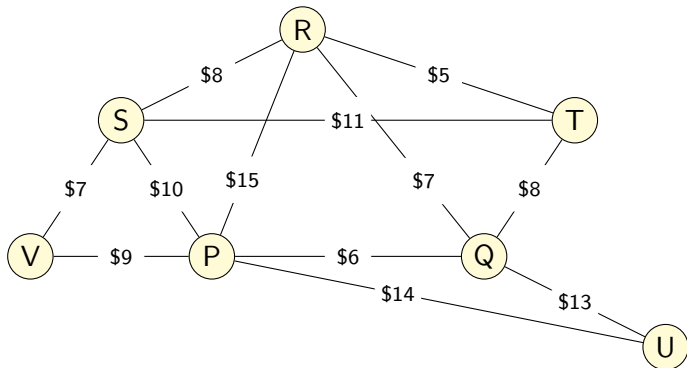
Step	Edge Considered	Forms Cycle?	Add to T?	Cost
1				
2				
3				
4				
5				
6				
7				
8				
9				



Algorithm 3 Kruskal's Algorithm

```

1: Input: A connected, undirected graph  $G = (V, E)$  with edge weights  $w(e)$ .
2: Initialization: Mark all edges as unused. Set  $T \leftarrow \emptyset$ .
3: repeat
4:   Select the cheapest unused edge in the graph.
5:   if adding the edge would create a cycle then
6:     Skip this edge.
7:   else
8:     Add the edge to  $T$ .
9:   end if
10: until  $T$  contains  $|V| - 1$  edges (i.e., a spanning tree has been formed).
11: Output: The set  $T$ , forming a minimum spanning tree of  $G$ .
  
```

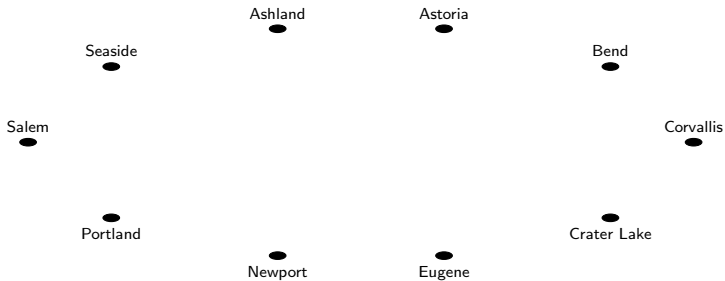


Step	Edge Considered	Forms Cycle?	Add to T?	Cost
1				
2				
3				
4				
5				
6				
7				
8				
9				

The power company needs to lay updated distribution lines connecting the ten Oregon cities below to the power grid. How can they minimize the amount of new line to lay?

	Ashland	Astoria	Bend	Corvallis	Crater Lake	Eugene	Newport	Portland	Salem	Seaside
Ashland	–	374	200	223	108	178	252	285	240	356
Astoria	374	–	255	166	433	199	135	95	136	17
Bend	200	255	–	128	277	128	180	160	131	247
Corvallis	223	166	128	–	430	47	52	84	40	155
Crater Lake	108	433	277	430	–	453	478	344	389	423
Eugene	178	199	128	47	453	–	91	110	64	181
Newport	252	135	180	52	478	91	–	114	83	117
Portland	285	95	160	84	344	110	114	–	47	78
Salem	240	136	131	40	389	64	83	47	–	118
Seaside	356	17	247	155	423	181	117	78	118	–

Using Kruskal's algorithm, we add edges from cheapest to most expensive, rejecting any that close a circuit. We stop when the graph is connected.

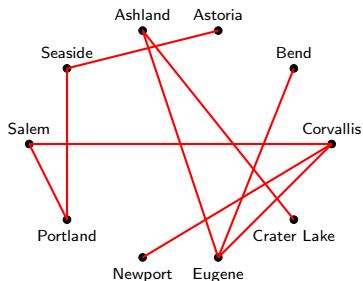


Extended Example: Oregon Power Grid

The power company needs to lay distribution lines connecting ten Oregon cities.

Cities with shortest connections:

- Seaside to Astoria: 17 miles
- Corvallis to Salem: 40 miles
- Portland to Salem: 47 miles
- Corvallis to Eugene: 47 miles
- Corvallis to Newport: 52 miles
- Portland to Seaside: 78 miles
- Ashland to Crater Lake: 108 miles
- Bend to Eugene: 128 miles
- Eugene to Ashland: 178 miles

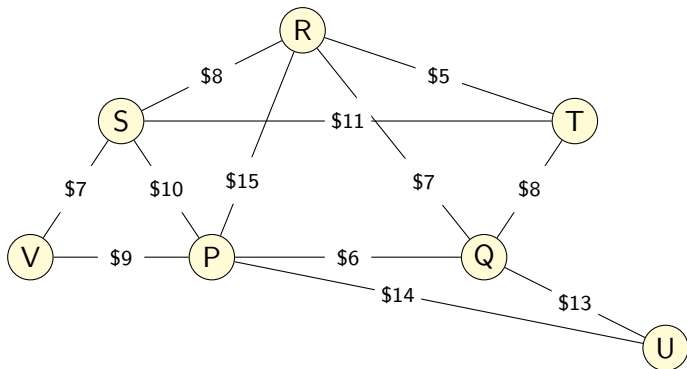


Total: 695 miles of cable

Key Takeaways

- A spanning tree is a connected subgraph with no cycles that includes all vertices
- A spanning tree of a graph with n vertices always has exactly $n - 1$ edges
- The minimum cost spanning tree (MCST) is the spanning tree with lowest total edge weight
- Kruskal's algorithm:
 - Sort edges by weight
 - Add edges in order of increasing weight
 - Skip edges that would create cycles
 - Stop when you have $n - 1$ edges
- Kruskal's algorithm always produces the optimal solution
- MCST has applications in network design, infrastructure planning, clustering, and more

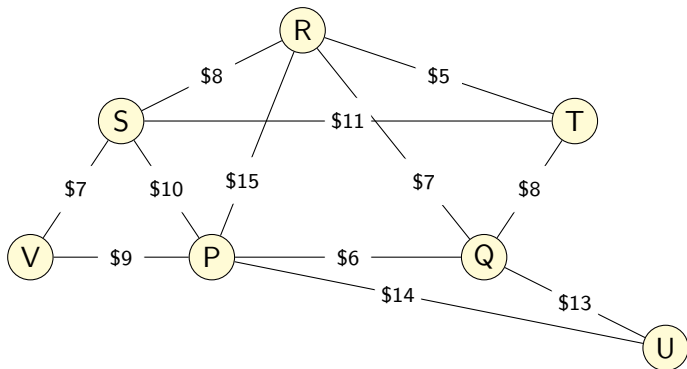
Prim's Algorithm



Algorithm 4 Prim's Algorithm

- 1: **Input:** A connected, undirected graph $G = (V, E)$ with edge weights $w(e)$.
- 2: **Initialization:** Choose an arbitrary start vertex $v_0 \in V$. Set $T \leftarrow \emptyset$ and $S \leftarrow \{v_0\}$.
- 3: **repeat**
- 4: From all edges (u, v) where $u \in S$ and $v \notin S$, select the edge with the minimum weight.
- 5: Add the selected edge to T and add the new vertex v to S .
- 6: **until** $|S| = |V|$ (i.e., all vertices are included).
- 7: **Output:** The set T , forming a minimum spanning tree of G .

Prim's Algorithm



Step	Edge Considered	Forms Cycle?	Add to T?	Cost
1				
2				
3				
4				
5				
6				
7				
8				

Greedy Algorithms for Minimum Spanning Trees

Prim's Algorithm and **Kruskal's Algorithm** both solve the *Minimum Spanning Tree (MST)* problem using a **greedy approach**.

What is a Greedy Algorithm?

A greedy algorithm builds a solution step-by-step, always choosing the **locally optimal** choice at each step, hoping this leads to a globally optimal solution.

Kruskal's Algorithm

- Sorts edges by weight.
- Adds the next lightest edge that doesn't form a cycle.
- Builds the MST by connecting components.

Prim's Algorithm

- Grows the MST from a starting vertex.
- At each step, adds the lightest edge that connects the tree to a new vertex.

Why Do Greedy Algorithms Work for MSTs?

Minimum Spanning Trees have two key properties that make greedy algorithms like **Prim's** and **Kruskal's** provably optimal:

Greedy-Choice Property

A globally optimal solution can be arrived at by making a series of locally optimal (greedy) choices.

- Choosing the lightest edge never prevents the optimal MST.
- Safe edges: The minimum-weight edge crossing any cut must be in some MST.

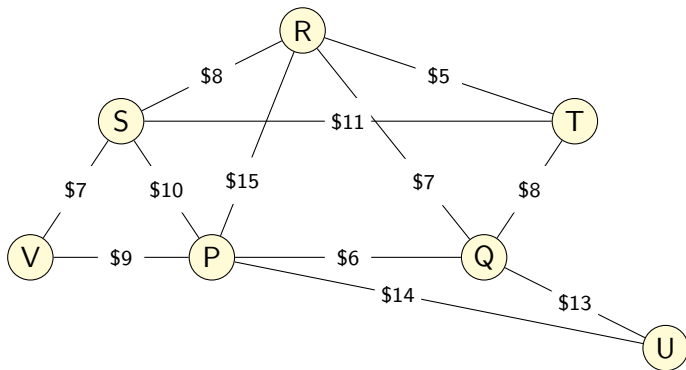
Optimal Substructure

An optimal solution to the problem contains within it optimal solutions to subproblems.

- Once we select a safe edge, we can recurse on the remaining graph.
- The MST is built up piece by piece, maintaining optimality at each step.

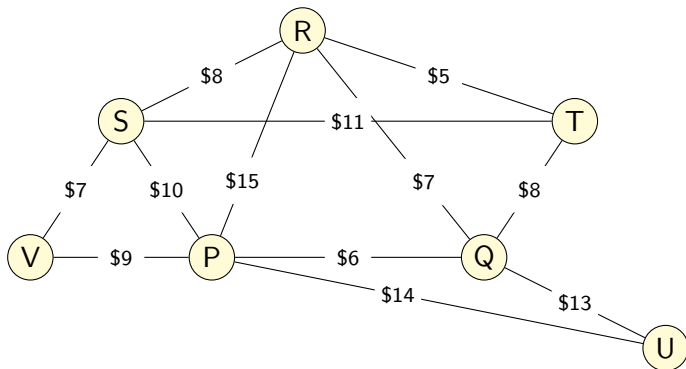
Optimality idea

The smallest weight edge must be in a MST



Optimality idea

The smallest weight edge across any cut must be in a MST



Resources

- YouTube Video: "Kruskal's algorithm to find a minimum weight spanning tree" by Kimberly Brehm
- Additional practice exercises available in the textbook

Questions?